

Deep Dive — Module 5: Python + Sentence Transformers (Embedding Service)

***Goal:** understand the component that turns text into meaning-vectors — what an embedding model is, how `all-MiniLM-L6-v2` works in practice, how the Flask wrapper exposes it over HTTP, and how it plugs into Spring Boot and Qdrant.*

Where it sits: This is the **AI core**. It's a small, single-purpose Python service: receive text, return a 384-number vector. Spring Boot calls it during *both* upload (to index) and search (to query). It is internal-only — never exposed to the internet.

1. What "embeddings" actually are

An **embedding model** maps text to a fixed-length list of numbers (a vector) such that texts with similar *meaning* get similar vectors. The model (`all-MiniLM-L6-v2`) was pre-trained on billions of sentence pairs, so it already "understands" language — you don't train anything; you just call `encode()`. This is what lets the system match an invoice to its near-duplicate even when the wording differs.

2. Loading the model (once)

```
pip install sentence-transformers flask flask-cors

from sentence_transformers import SentenceTransformer
model = SentenceTransformer('all-MiniLM-L6-v2') # downloads ~200 MB weights once, loads into f
```

The model is loaded **a single time at service startup** and held in memory. Loading is the slow part (seconds); after that each `encode()` is ~1 ms on CPU. Loading per-request would be catastrophic for latency — so the Flask app creates `model` at module scope, not inside the route.

3. Encoding text into a vector

```
vector = model.encode("Invoice from Acme Corp, total USD 1250.75")
vector.shape      # (384,) → numpy array of 384 floats
vector.tolist()   # convert to a plain Python list for JSON
```

The output is always 384 numbers for this model — the dimension Qdrant's collection is configured for. `.tolist()` is required because numpy arrays aren't directly JSON-serializable.

4. Why similar texts → similar vectors

```
from sklearn.metrics.pairwise import cosine_similarity
v1 = model.encode("Invoice from Acme Corp $1250")
v2 = model.encode("Bill from Acme Corp 1250 USD")    # same meaning, different words
v3 = model.encode("Bank statement HDFC Q1 2026")    # different topic
cosine_similarity([v1],[v2]) # ≈ 0.94 → strong match
cosine_similarity([v1],[v3]) # ≈ 0.21 → unrelated
```

The model places semantically related text close together in 384-dimensional space; cosine similarity (Module 2) measures that closeness. This is the mathematical basis for the whole product.

5. The Flask API wrapper

Flask is a minimal Python web framework. It exposes the model over HTTP so the Java backend can call it.

```

from flask import Flask, request, jsonify
from sentence_transformers import SentenceTransformer

app = Flask(__name__)
model = SentenceTransformer('all-MiniLM-L6-v2') # loaded ONCE at startup

@app.route('/embed', methods=['POST'])
def embed():
    text = request.get_json().get('text', '')
    return jsonify({'embedding': model.encode(text).tolist()})

@app.route('/embed-batch', methods=['POST'])
def embed_batch():
    texts = request.get_json().get('texts', [])
    return jsonify({'embeddings': model.encode(texts).tolist()}) # encode list = faster

@app.route('/health', methods=['GET'])
def health():
    return jsonify({'status': 'UP'})

if __name__ == '__main__':
    app.run(host='0.0.0.0', port=5000)

```

- `host='0.0.0.0'` so it's reachable from other containers on the Docker network.
- `/embed` for single chunks; `/embed-batch` for many at once (fewer HTTP round-trips, and the model vectorizes the batch internally).
- `/health` lets the backend's health check confirm the service is up.

6. Request flow through the service

```

Spring Boot → POST http://localhost:5000/embed
               { "text": "Invoice from Acme Corp INV-2048 Total $1250.75" }
Flask         → model.encode(text) → numpy (384,) → .tolist()
               ← { "embedding": [0.023, -0.118, 0.204, ... 384 floats.] }
Spring Boot → stores the 384 numbers as a Qdrant vector point

```

The *same* path runs for search, except the resulting vector is used to query Qdrant instead of being stored.

7. Choosing the model: all-MiniLM-L6-v2

Dimensions: 384 Size: ~200 MB CPU: ~750 q/s License: Apache 2.0 (free)
Quality: good for semantic search of short/medium financial text

Alternative – all-mpnet-base-v2:

Dimensions: 768 (more precise) CPU: ~170 q/s (≈4× slower) Quality: higher

The project picks MiniLM because it's fast on CPU (no GPU needed for on-prem), small, free, and "good enough" for invoice-style text. The PRD flags a possible upgrade to the 768-dim model if hardware allows — note that switching changes Qdrant's collection size to 768 *and* requires re-embedding the entire corpus and the pgvector column width.

8. Production considerations

- **Single instance** is fine locally; for higher throughput run multiple workers (gunicorn) or a GPU.
 - **Batching** (`/embed-batch`) is the biggest easy win — embedding 50 chunks in one call beats 50 calls.
 - **Determinism**: the same text always yields the same vector, so re-embedding is safe and idempotent.
 - **Containerized** via its own Dockerfile (Module 6); model weights download at build or first run.
 - **Internal-only**: no auth on the service itself because it's never internet-exposed; it trusts the backend on the private network.
-

9. Common pitfalls

- **Loading the model inside the route** → multi-second latency per request. Load once at startup.
- **Returning numpy directly** → JSON serialization error; always `.tolist()`.
- **Dimension drift** → if you change models, the 384 ≠ 768 mismatch breaks Qdrant upserts silently or loudly.
- **One HTTP call per chunk** → use `/embed-batch` to avoid round-trip overhead.

- **Assuming GPU** → the on-prem default is CPU; size expectations accordingly (~750 q/s).
 - **Sending huge text in one chunk** → embeddings represent ~a paragraph well; that's why text is chunked first (Module 9).
-

10. Practice exercises

`pip install sentence-transformers flask` in a venv.

1. Load the model and print `model.encode("hello world").shape`. Confirm it's `(384,)`.
 2. Encode three sentences (two similar, one different) and compute pairwise cosine similarity; verify the pattern.
 3. Write the Flask app with `/embed`, `/embed-batch`, and `/health`. Run it and `curl` each endpoint.
 4. Compare timing: 20 separate `/embed` calls vs one `/embed-batch` of 20 texts.
 5. Confirm determinism: encode the same text twice and check the vectors are identical.
 6. Write the Java-side parsing logic (pseudo-code) that converts the JSON `embedding` list into a `float[384]`.
 7. (Stretch) Wrap the service in a Dockerfile (`FROM python:3.10-slim`) and run it on port 5000.
 8. (Stretch) Swap in `all-mpnet-base-v2`, observe the `(768,)` shape, and list everything else that would need to change.
-

11. Self-check questions

- What is an embedding, and why does similar text produce similar vectors?
 - Why is the model loaded once at startup instead of per request?
 - Why must `.tolist()` be called before returning the vector?
 - Why does the collection's dimension (384) have to match the model?
 - When would you use `/embed-batch` over `/embed`?
 - What are the trade-offs of moving to the 768-dim model?
 - Why does this service need no authentication of its own?
-

12. Glossary

- **Embedding / vector** — numeric representation of meaning (384 dims).
 - **Sentence Transformers** — Python library of pre-trained embedding models.
 - **all-MiniLM-L6-v2** — the project's model: 384-dim, fast, free.
 - **encode()** — turns text into a vector.
 - **Flask** — lightweight Python web framework wrapping the model.
 - **Batch encoding** — embedding many texts in one call.
 - **Cosine similarity** — closeness metric between two vectors.
-

Navigation: [← Module 4 Spring Boot](#) | [Index](#) | [Module 6 Docker →](#)